

A Taxonomy of Failure Modes in a Sub-3B Parameter Agentic AI System on Edge Hardware

Martin Cote – Independent Researcher (martin.cote@gmail.com)

April 03, 2026

Abstract

Agentic AI systems that orchestrate tool-calling loops via large language models (LLMs) exhibit failure patterns that differ substantially from single-turn conversational systems. I present an empirically derived taxonomy of failure modes observed in `bouc.io`, a self-hosted agentic platform running Qwen3.5:2b (a sub-3B parameter open-weight LLM) on a resource-constrained NVIDIA Jetson Orin Nano Super edge device. Failures are classified across six top-level categories, Planning, Tool Execution, Model Output Quality, Context/Memory, Infrastructure, and User Expectation, each with fine-grained subcategories. The taxonomy is derived from 200 runs collected via semi-naturalistic opportunistic sampling combining purposive test queries and organic platform use sessions, rated by the system operator using a thumbs-up/down feedback mechanism with free-text comments. I analyze failure frequency, co-occurrence with tool usage patterns, and correlations with run duration and token consumption. The findings highlight that planning failures, particularly looping and wrong-tool selection, account for the majority (47%) of coded failures, that negative-feedback runs consume 45.8% more tokens than successful ones, and that no technical infrastructure failures were observed on the Jetson hardware across this dataset.

1. Introduction

Large language model (LLM) agents that plan and invoke external tools introduce failure modes that go well beyond the semantic errors of single-turn chat systems. Prior work has catalogued failures in cloud-hosted frontier models [1], [2], but the behavior of *sub-3B* parameter open-weight models in agentic settings on resource-constrained hardware is poorly understood as far as I can tell. Small models have been shown to achieve only 20–40% accuracy on complex agentic tasks [3], yet the specific failure patterns, and their operational triggers, have not been systematically characterized in a production deployment.

I address this gap with a taxonomy derived from semi-naturalistic opportunistic sampling of `bouc.io` [4], a self-hosted agentic platform running Qwen3.5:2b [5] on an NVIDIA Jetson Orin Nano Super [6]. Rather than constructing an artificial benchmark, I exploit in-situ user feedback (thumbs-up/down with free-text comment) collected over 200 runs spanning 199 rated interactions.

2. Related Work

ReAct [1] introduced the interleaved reason-act paradigm that underlies most current tool-calling agents. ToolFormer [2] showed that LLMs can learn to call APIs self-supervised, but analysis focused on correctness rather than failure characterization. AgentBench [7] benchmarks agent performance across tasks, noting that smaller models suffer disproportionate plan-

ning failures. HuggingGPT [8] and similar orchestration frameworks identify tool-selection errors as a dominant failure class. ToolLLM [9] benchmarks tool-calling across 16,000+ real-world APIs, finding that even large models struggle with multi-hop chains — a pattern my data confirms at sub-3B scale. Sumers et al. [10] provide a comprehensive taxonomy of cognitive architectures for language agents, within which the planner-executor pattern of `bouc.io` corresponds to a plan-then-act decomposition. MemGPT [11] addresses context limitations via hierarchical memory management; the failure modes it targets (C-MM, C-OV) remain latent in our deployment due to current memory system constraints (see Discussion). My work differs in three ways: (1) it uses a sub-3B model, (2) it is grounded in a production-like deployment on edge GPU hardware, and (3) failure labels come from the operator rather than an automated evaluator.

3. Methodology

3.1 System Overview

`bouc.io` implements a two-phase planner-executor agent [4]. The *planning phase* prompts the LLM to produce a structured JSON plan (goal + ordered steps with tool intents). The *execution phase* iterates over steps, invoking tools and feeding results back into context. Fourteen tools are available: `web_search`, `calculator`, `http_request`, `echo`, `fetch_page`, `json_query`, `text_extract`, `memory_search`, `bash`, `read_file`, `write_file`, `glob_files`, `list_dir`, and `time_date`. Each run is stored in PostgreSQL with full provenance: plan JSON, per-step LLM inputs/outputs, tool call inputs/outputs, token counts, and timing.

3.2 Observed Performance

Table 1: Observed operational performance measurements.

Service	Metric	Value	Unit
Agent	Avg. planner latency (Qwen3.5:2b)	10565 / 12565	ms
	Avg. tool-call round-trip	1879	ms
	Memory retrieval p50 / p95	1305 / 12347	ms
	Distillation pipeline p50 / p95	15.0 / 40.3	s
Chatbot	Avg. response latency (w/ thinking)	8563 / 189731	ms
	Avg. tokens per response	1399	tokens

† Chatbot latency measured with thinking enabled; p95 driven by reasoning chain length variance, not I/O or retrieval overhead.

3.3 Data Collection

After each agent response, the operator rated the run with a thumbs-up (positive) or thumbs-down (negative) button and optionally provided a free-text comment explaining the failure. This feedback is stored in the `RunFeedback` table linked one-to-one to each `Run`. Data was extracted in-

crementally via the SQL query in `data/extract_runs.sql`, joining `Run`, `Assignment`, `Message`, `ToolCall`, `RunStep`, and `RunFeedback`. Each extraction batch is saved as a dated CSV (`runs_raw_YYYY-MM-DD_HH-MM-SS.csv`); the Rmd loads and combines all batches at render time. Queries were a mix of deliberate tool-exercising prompts (see Appendix) and sessions arising from routine platform use.

The resulting dataset contains **200 runs** of which **199** are rated (**148 positive**, **51 negative**). An additional **0** runs ended in technical failure (`status = 'failed'`).

3.4 Coding Procedure

Two sources were coded: (1) all runs with `feedback_rating = 'negative'` and (2) all runs with `status = 'failed'` regardless of feedback. Coding was performed by the operator in-place within each dated batch CSV after extraction. Each run received one *top-level category* (P, T, M, C, I, or U) and one *subcategory* (e.g., P-LP for planning loop). In cases of multiple failures, the primary cause was coded. **51 runs** received failure codes, corresponding to all 51 negative-feedback runs in this dataset (no technical failures were observed).

4. Taxonomy

Table 1 presents the full taxonomy. Six top-level categories were identified, with 20 subcategories total.

Table 2: Failure taxonomy (6 top-level categories, 20 subcategories).

Code	Description
P	Planning failures
P-NJ	No valid JSON plan produced
P-OD	Over-decomposition (too many steps)
P-WT	Wrong tool selected in plan
P-LP	Planning loop / duplicate plan
T	Tool execution failures
T-NF	Tool not found (registry mismatch)
T-TO	Tool execution timeout
T-PE	Tool output parsing error
T-EX	Tool raised exception
T-NE	Tool not executed (execution system skipped step)
M	Model output quality
M-HL	Hallucinated fact or tool name
M-RP	Repetitive / looping response
M-INC	Incomplete response (cut off)
M-RF	Model refusal
C	Context / memory
C-MM	Wrong memories injected
C-OV	Context overflow (prompt truncated)
I	Infrastructure
I-OOM	Out-of-memory (Jetson OOM)
I-NET	Network / Redis error
I-DB	Database error
U	User expectation gap
U-MI	Misunderstood intent
U-IN	Insufficient depth or detail

5. Results

Across all completed runs, the median total token consumption was 1.1248×10^4 tokens (IQR: 8246 – 1.369175×10^4). Negative-feedback runs consumed a median of 1.2884×10^4 tokens versus 8834.5 for positive runs, suggesting that failure-prone interactions involve longer context accumulation across planning and execution phases.

5.1 Overall Run Quality

Figure 1 shows the distribution of run outcomes across the full dataset of 200 runs. Runs are classified as *positive feedback*, *negative feedback*, *failed* (`status = 'failed'`), or *no feedback*

(unrated).

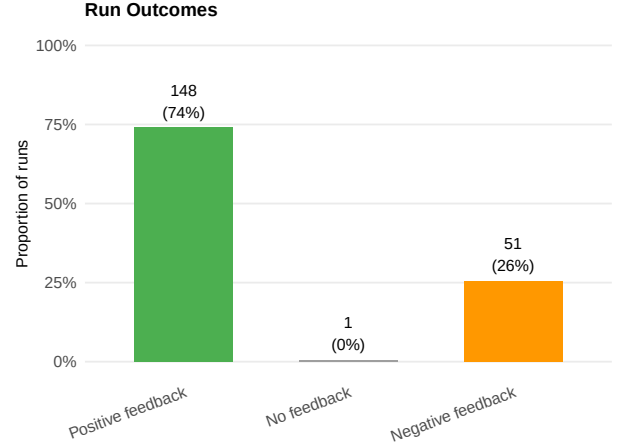


Figure 1: Run outcome distribution.

5.2 Failure Category Frequency

Figure 2 shows the share of coded failures attributed to each of the six top-level taxonomy categories. Only the 51 runs that received a failure code contribute to this chart.

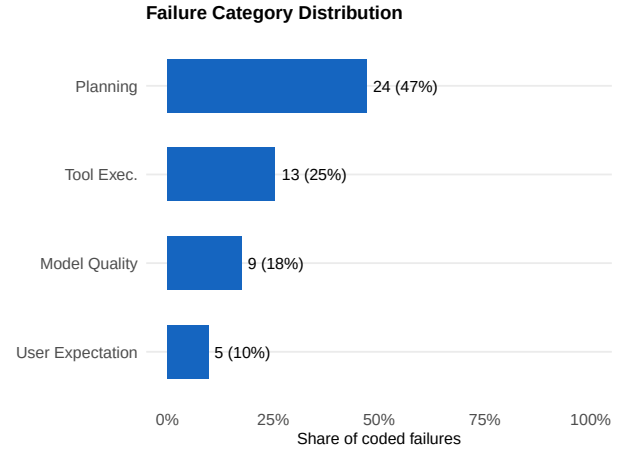


Figure 2: Frequency of failure categories among coded runs.

5.3 Run Duration by Outcome

Figure 3 compares run duration in seconds across outcome categories. Longer durations in negative or failed runs reflect additional planning and execution cycles before the agent terminates or the user abandons the interaction.

5.4 LLM Call Count Distribution

Figure 4 shows the distribution of LLM calls per run, stacked by outcome. A high LLM call count for negative runs is a proxy signal for planning loops (P-LP), where the planner repeatedly regenerates steps before the executor detects the cycle.

5.5 Token Usage by Outcome

Figures 5 and 6 quantify token consumption by outcome. Negative-feedback and failed runs are expected to consume more tokens due to additional context accumulation across repeated planning and execution cycles.

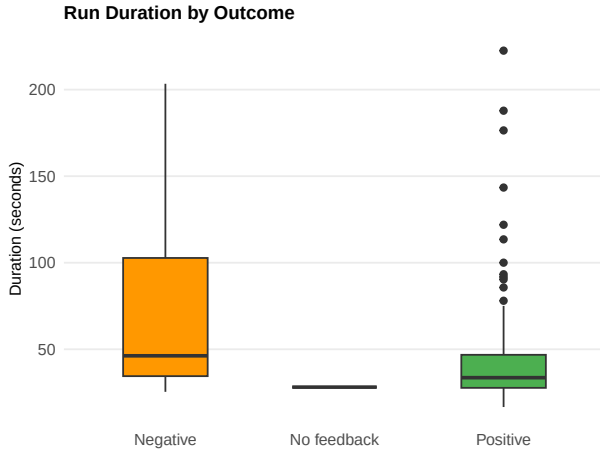


Figure 3: Run duration (s) by outcome category. Horizontal bars show medians.

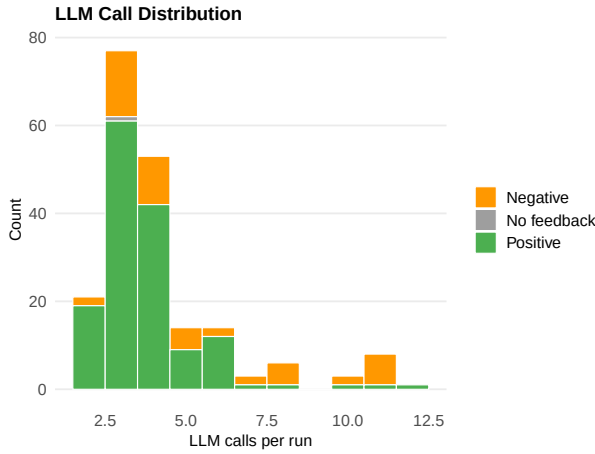


Figure 4: Distribution of LLM calls per run by outcome.

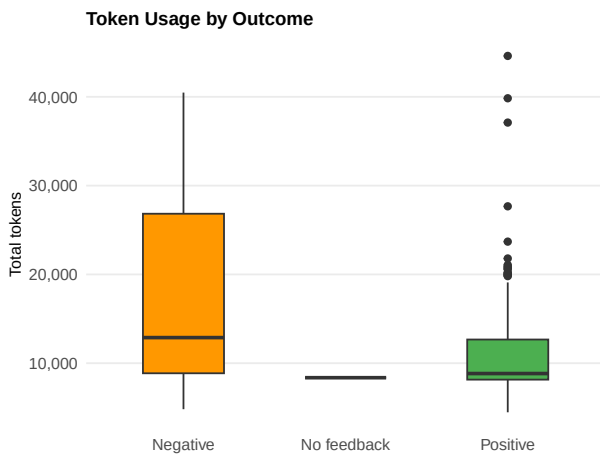


Figure 5: Total tokens per run by outcome. Bars show medians.

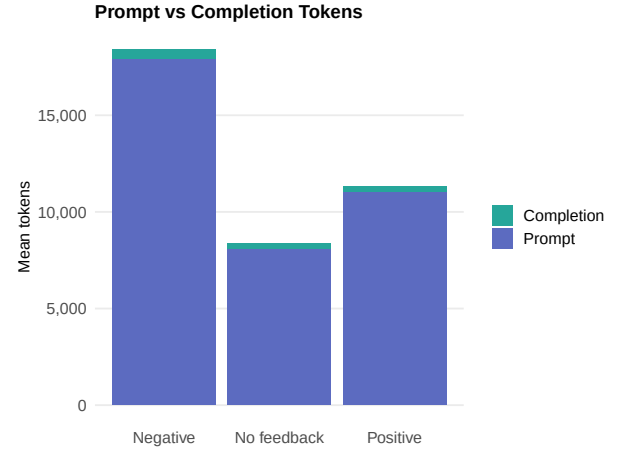


Figure 6: Mean prompt vs completion tokens by outcome.

5.6 Subcategory Breakdown

Table 2 presents failure counts at the subcategory level. This view identifies which specific failure modes within each top-level category are most prevalent, enabling targeted mitigation prioritization.

Table 3: Failure subcategory counts.

Category	Subcategory	Count
M	M-HL	4
M	M-RP	4
M	M-INC	1
P	P-LP	12
P	P-WT	9
P	P-OD	2
P	P-NJ	1
T	T-PE	7
T	T-NE	3
T	T-EX	2
T	T-TO	1
U	U-IN	3
U	U-MI	2

5.7 Tool Usage and Failure Co-occurrence

Figure 7 contrasts tool invocation frequency between runs with positive outcomes (“OK”) and runs with negative feedback or failures (“Problematic”). Tools that appear disproportionately in problematic runs may indicate model difficulty with specific tool schemas or output formats.

5.8 Failures by Model

Figures 8 and 9 present outcome and failure-category distributions broken down by model. All runs in the current dataset used `qwen3.5:2b`; cross-model comparison will populate automatically as additional model deployments are instrumented.

6. Discussion

Of the 199 rated runs, 25.6% received negative feedback and 0% ended in technical failure. The most frequent failure category was **Planning** (47% of coded failures). These results are consistent with prior reports that sub-3B models struggle with structured output generation and multi-step reasoning [3], [7]. Several patterns warrant highlighting. First, planning loops (P-LP) are the single dominant failure subcategory, occurring when the planner re-generates the same step or emits duplicate tool intents in successive LLM calls. This drives disproportionately

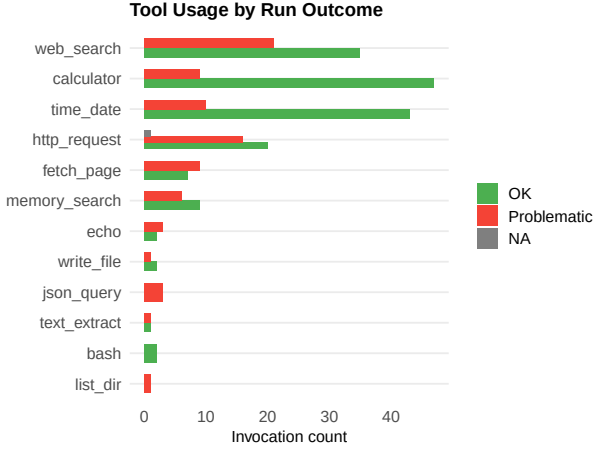


Figure 7: Tool invocation frequency for positive vs negative runs.

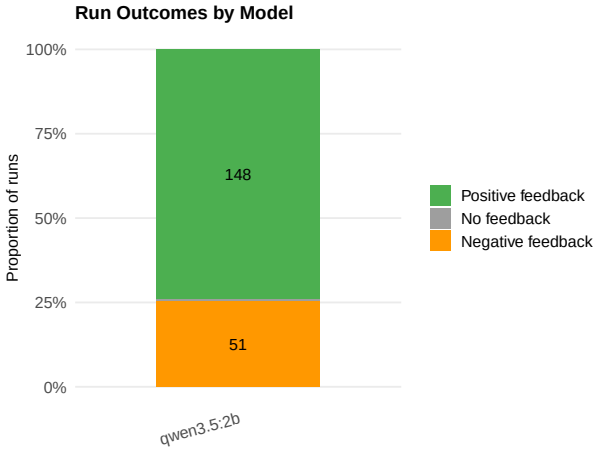


Figure 8: Run outcome distribution by model.

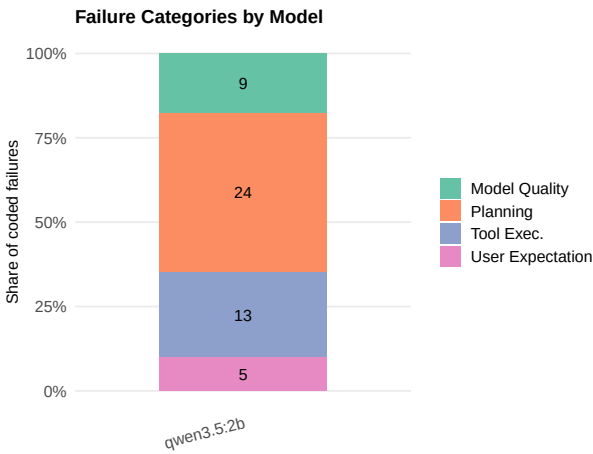


Figure 9: Failure category distribution by model (coded runs only).

high token consumption: negative-feedback runs consumed a median of 1.2884×10^4 tokens versus 8834.5 for positive runs — a **46% increase** (Mann-Whitney U, $W=,5394$, $p,<0.001$) — as the loop accumulates context before the executor detects the cycle. Task complexity is a likely confound: more demanding queries may drive both higher token use and greater failure likelihood. Second, tool output parsing errors (T-PE) represent the dominant tool-execution failure, arising when the model returns malformed JSON in a tool call argument rather than within the structured plan. This is consistent with small-model limitations in maintaining strict output schemas under long context [3]. Third, wrong-tool selection (P-WT) occurs when the planner assigns a `web_search` intent to a query that the `calculator`, `http_request`, `web_search` or `time_date` tool could resolve deterministically, increasing latency and introducing retrieval noise.

Notably, no infrastructure failures (I-OOM, I-NET) were observed across 200 runs, suggesting that the hardware-level stability of the Jetson Orin Nano Super is adequate for the observed query load, and that the llama3.2-to-qwen3.5:2b model migration successfully resolved the out-of-memory crashes documented in earlier operations. It is important to note, the tests were done under a single user workload, allowing plenty of capacity to process each request in sequence.

The absence of Context/Memory (C) failures warrants explanation. The platform’s memory distiller uses the same Qwen3.5:2b model to evaluate and extract memories, applying a confidence threshold of 0.5. In practice this produces few high-quality extractions per run, resulting in sparse memory injection across sessions. With minimal memories in context, retrieval failures (C-MM) and context overflow (C-OV) are rarely triggered under current operating conditions. This is a system maturity limitation rather than evidence that C-category failures are absent in principle, as a more capable memory pipeline would be expected to surface this category [11].

The in-situ feedback mechanism proved effective for semi-naturalistic data collection: the operator naturally left short comments that described not just *what* failed but *why*, providing richer signal than a binary label alone. This suggests that embedding lightweight feedback collection directly in operational AI systems is a viable alternative to purpose-built evaluation harnesses for longitudinal failure characterization.

Regarding the model-level analysis: all 200 runs in this dataset used a single model (`qwen3.5:2b`), so cross-model comparison charts reflect a single stratum. Comparative analysis across models is reserved for future work as additional model deployments are instrumented.

6.1 Limitations

This study has several limitations. First, all failure codes were assigned by a single coder (the system operator, i.e. myself), introducing potential for confirmation bias. No inter-rater reliability measure (e.g., Cohen’s kappa) was computed; internal consistency was partially mitigated by applying the codebook in a single continuous session with explicit decision rules for ambiguous cases. Second, the dataset covers a single model (Qwen3.5:2b) on a single deployment; findings may not generalise to other sub-3B models or hardware configurations. Third, data collection blends purposive test queries with organic use sessions, limiting claims about the representativeness of failure rates relative to unconstrained deployment.

7. Conclusion

I have presented a six-category, 20-subcategory taxonomy of failure modes for a sub-3B agentic AI system on edge GPU hardware, grounded in 200 semi-naturalistic runs on a single-node Jetson Orin Nano Super deployment. Planning failures, particularly looping (P-LP) and wrong-tool selection (P-WT), account for 47% of coded failures and are the primary driver of elevated token consumption in problematic runs. Tool output parsing errors (T-PE) represent the dominant execution-layer failure, consistent with small-model limitations in strict schema adherence. Infrastructure failures were absent in this dataset, validating the hardware and model selection decisions made during platform iteration. The taxonomy provides a structured vocabulary for discussing, prioritizing, and mitigating agentic system failures in resource-constrained deployments. The dataset is incremental and will grow as the platform is used; the analysis pipeline is designed to re-render automatically as new rated runs are collected and coded. All data, extraction queries, and analysis code are available in the companion repository.

Appendix: Test Query Corpus

The following queries were submitted to the agent during data collection. Queries were chosen to cover the main tools available (`web_search`, `calculator`, `time_date`, `echo`, `http_request`) as well as multi-step chaining, ambiguous intent, and out-of-scope requests likely to surface model quality and user expectation failures.

Single-tool — web search

- Search for the weather in Paris.
- Who is the current Prime Minister of Canada?
- What is the capital city of Australia?
- What is the population of Tokyo?
- What is the current stock price of Apple?

Single-tool — calculator

- What is 2+2, then multiply that by 3, then add 10 to it.
- What is 15% of 847?
- What is the square root of 144, divided by 3?
- If I invest \$1,000 at 5% annual interest for 10 years, what is the final amount?

Single-tool — time / date

- What time is it in Tokyo?
- What time is it right now in London?
- What day of the week will January 1st, 2027 fall on?
- How many days until Christmas?

Multi-step / tool chaining

- How long is the drive between Montreal, QC and New York City, NY?
- What is today's date? Then tell me what date it will be in 90 days.
- Search for the speed of light in km/s, then calculate how far light travels in 10 minutes.

Ambiguous / edge cases

- Tell me a fun fact.
- What is the meaning of life?
- Summarize the history of the internet in 3 bullet points.

Out-of-scope / user expectation

- Write me a Python function to sort a list.
- Translate “hello, how are you?” into French, Spanish, and Japanese.

Platform source code and infrastructure modules for `bouc.io` are maintained in a series of private GitLab repositories. Blog posts documenting each major system milestone and specific technical components are published at <https://www.martincote.ca/blogs.html>.

Data & Code Availability. *The R Markdown source, performance metrics CSV, and collection scripts for this paper are openly available at <https://github.com/MartinCote1978/agent-failure-taxonomy-research>*

AI Usage Disclosure. *This document was created with assistance from AI tools. The content has been reviewed and edited by a human. For more information on the extent and nature of AI usage, please contact the author.*

References

- [1] S. Yao *et al.*, “ReAct: Synergizing reasoning and acting in language models,” *arXiv preprint arXiv:2210.03629*, 2022.
- [2] T. Schick *et al.*, “Toolformer: Language models can teach themselves to use tools,” *arXiv preprint arXiv:2302.04761*, 2023.
- [3] Z. Tang *et al.*, “Small language models: Survey, measurements, and insights,” *arXiv preprint arXiv:2409.15790*, 2024.
- [4] M. Cote, “Bouc.io: A self-hosted agentic AI platform on edge kubernetes with open-weight LLMs.” 2026.
- [5] Qwen Team, “Qwen2 technical report,” *arXiv preprint arXiv:2407.10671*, 2024.
- [6] NVIDIA Corporation, “NVIDIA Jetson Orin Nano series system-on-module datasheet.” 2023.
- [7] X. Liu *et al.*, “AgentBench: Evaluating LLMs as agents,” *arXiv preprint arXiv:2308.03688*, 2023.
- [8] Y. Shen, K. Song, X. Tan, D. Li, W. Lu, and Y. Zhuang, “HuggingGPT: Solving AI tasks with ChatGPT and its friends in HuggingFace,” *arXiv preprint arXiv:2303.17580*, 2023.
- [9] Y. Qin *et al.*, “ToolLLM: Facilitating large language models to master 16000+ real-world APIs,” *arXiv preprint arXiv:2307.16789*, 2023.
- [10] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths, “Cognitive architectures for language agents,” *arXiv preprint arXiv:2309.02427*, 2023.
- [11] C. Packer, V. Fang, S. G. Patil, K. Moon, S. Wooders, and J. E. Gonzalez, “MemGPT: Towards LLMs as operating systems,” *arXiv preprint arXiv:2310.08560*, 2023.